Real-world (conflicting requirements design) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order against your own written design before reading ahead.

- Hint 1 — What to ask, and the obvious first (wrong) instinct
- Hint 2 — The plan, and almost the whole design

Hint 1 — What to ask, and the obvious first (wrong) instinct

Name the conflict explicitly, in writing, before you design a single agent: "fast and fully autonomous" and "a human reviews everything" cannot both be fully true at once. A reasonable way to reconcile them without more information is a **tiered-risk routing pattern** — a conditional routing pattern, the same family as Doc11's router patterns: low-risk work (routine topics, well-sourced, high fact-check confidence) auto-publishes; higher-risk work (contested topics, anything touching legal/financial/medical claims, low fact-check confidence) routes to a human reviewer.

This isn't "solving" the disagreement on your own authority — it's a specific *proposed* compromise design, and it has to be flagged explicitly as an assumption, with the actual question you'd ask stakeholders written down next to it: "where should the auto-publish / requires-review line sit, and who signs off on that line?"

What a senior reviewer pushes on:

The trap at this level is over-engineering to "please everyone" — bolting on every stakeholder's individual ask (a full audit trail for legal, an autonomous fast-path for the speed request, a hard cost ceiling for finance, a manual override switch for every edge case anyone can imagine) produces something so complex nobody can explain or maintain it — and it still hasn't actually resolved the underlying disagreement about who owns the risk-tier decision. It just buries the disagreement inside code where nobody will notice it wasn't actually settled.

The senior move is the opposite of "build everything": design the smallest system that resolves the conflict through one explicit, named decision (here, the risk-tier cutoff), present that decision plainly as a stated assumption, and ask the actual stakeholders to confirm or correct it — instead of silently encoding your own guess as if it were already agreed policy. If you're pushed on why you didn't just build in every stakeholder's individual request, the answer is that unreviewed, unauthorized policy

calls baked into code are exactly the kind of hidden decision a real system-design review exists to catch.

Hint 2 — The plan, and almost the whole design

```
Step 1 -- name every conflict explicitly (example shape):
    "fast, fully autonomous" (stakeholder A)
        vs.
    "human review of everything, no exceptions" (stakeholder B)
        vs.
    "minimize API/compute cost" (stakeholder C)

Step 2 -- propose a reconciling pattern, not a guess buried in code:
    tiered-risk conditional routing (Doc11 routing family)
    risk score = f(topic sensitivity, source confidence, fact-check score)
    low risk  -> auto-publish   (serves "fast")
    high risk -> human review   (serves "safe")

Step 3 -- flag it, don't silently decide it:
    write the proposed risk-tier cutoff down as a stated assumption
    write the exact question for stakeholders: "where should this line
        sit, and who owns moving it?"

Step 4 -- address the requirement nobody reconciled yet (cost):
    add a cost cap/guardrail as its own explicit piece, not folded
    silently into the routing logic
```

Hardened for a senior review:

```
Same 4 steps, plus the discipline of resisting "build everything":

For every stakeholder ask, before adding a component for it, ask:
    does this genuinely serve a real requirement, or is it here so I
    can say "I addressed everyone's concern" without actually deciding
    anything?

Concretely reject, on purpose, and say so out loud in the design:
    - a manual override switch for every conceivable edge case
      (this doesn't resolve the risk-tier question, it just adds a
      lever nobody has agreed how to use)
    - a fully separate approval workflow bolted on per-stakeholder
      instead of one shared risk-tier decision everyone routes through

Present the design as:
```

```
1. here are the conflicts, named plainly
2. here is the proposed resolution (tiered routing) and exactly
   which assumption it rests on
3. here is the one open question stakeholders need to actually
   answer before this ships as policy, not just as code
```

Full solution: Show me the solution